

Zero-Redundancy Extraction Algorithm with FFT Integration and Noise-Tolerant Deobfuscation

Professional Algorithm Design: End-to-End Development, Benchmarking, and Implementation in 2026

Introduction

In the rapidly evolving landscape of computer science and artificial intelligence, **algorithm design and implementation** remain foundational to technological progress. Algorithms underpin everything from data processing and optimization to machine learning, quantum computing, and autonomous systems. As of 2026, the field has witnessed a convergence of classical and modern paradigms, with advances in automation, explainability, benchmarking, and deployment frameworks. This report provides a comprehensive, professional-grade exploration of the end-to-end process of algorithm development, from conceptualization and formal specification to implementation, benchmarking, and operationalization. It integrates current legal, ethical, and safety considerations, and draws on the latest academic and industrial trends, including quantum-classical comparisons, reproducibility, and open-source licensing. The report is structured to guide the reader through the **full lifecycle of algorithm development**, referencing recent literature, best practices, and real-world case studies. It is intended for researchers, engineers, and decision-makers seeking a deep, actionable understanding of algorithmic innovation in 2026.

1. Legal and Safety Constraints on Algorithmic Implementations

1.1 Intellectual Property and Licensing

The legal landscape for algorithmic IP is complex and evolving. In the EU and US, **algorithms as abstract ideas are generally not patentable** unless embedded in a computer-implemented invention (CII) that demonstrates technical effect^[2]. The European Patent Office (EPO) and US Patent and Trademark Office (USPTO) have both clarified that algorithms can be patented if they are part of a technical solution, such as a neural network for heart monitoring, but not if they are mere mathematical methods^[1]. Trade secrets remain the most common protection, but this approach raises transparency and accountability concerns, especially in high-stakes domains like healthcare and criminal justice.

Open-source licensing is now the norm for many algorithmic libraries and frameworks. The choice of license (MIT, Apache 2.0, GPL, etc.) has significant implications for commercial use, patent protection, and compliance^[4]. For example, the Apache 2.0 license includes explicit patent

grants, making it preferable for enterprise deployments, while the GPL enforces copyleft, requiring derivative works to remain open source.

Table 1: Comparison of Major Open Source Licenses for Algorithms

License	Permissive	Copyleft	Patent Protection	Commercial Use	Attribution Required
MIT	Yes	No	No	Yes	Yes
Apache 2.0	Yes	No	Yes	Yes	Yes
GPL	No	Strong	Yes	Yes (with conditions)	Yes
BSD	Yes	No	No	Yes	Yes

The choice of license affects not only legal compliance but also the **adoption, collaboration, and sustainability** of algorithmic projects. Modern best practices recommend clear documentation of licensing, explicit patent clauses where relevant, and proactive compliance audits, especially for organizations integrating open-source and proprietary components^[4].

1.2 Safety, Security, and Compliance

Algorithmic safety is paramount in domains such as autonomous vehicles, healthcare, and finance. Regulatory frameworks like the **EU AI Act** and sector-specific standards (e.g., ISO/IEC 23894 for AI risk management) require rigorous documentation, risk assessment, and explainability^[6]. Key safety considerations include:

- **Robustness to adversarial inputs** (especially in machine learning and AI)^[8]
- **Data privacy and compliance** with regulations such as GDPR and HIPAA
- **Transparency and explainability** to facilitate auditing and user trust^[10]
- **Fail-safe and fallback mechanisms** in critical systems

Security is increasingly addressed through **secure coding practices**, formal verification, and adversarial testing. For example, adversarial robustness in deep learning is now a major research area, with techniques such as adversarial training, verification, and explainability being actively developed and standardized^[8].

2. User Intent Clarification and Scope Definition

2.1 Requirements Engineering

A professional algorithm development process begins with **precise requirements engineering**. This involves:

- **Stakeholder analysis:** Identifying all parties affected by the algorithm, including end-users, operators, and regulatory bodies.

- **Functional requirements:** What the algorithm must accomplish (e.g., sort data, classify images, optimize routes).
- **Non-functional requirements:** Constraints on performance, interpretability, scalability, security, and compliance.
- **Success criteria and benchmarks:** Defining what constitutes a "good" solution, including accuracy, speed, resource usage, and fairness.

Recent literature emphasizes the importance of **domain-specific context** and the construction of a "realm" for the algorithm specification, which includes all necessary background knowledge, data semantics, and operational constraints^[11].

2.2 Problem Formalization

Formalizing the problem is critical for reproducibility and clarity. This typically involves:

- **Mathematical modeling:** Expressing the problem in terms of variables, objective functions, and constraints (see Section 5).
- **Input/output specification:** Defining the data types, formats, and expected outputs.
- **Assumptions and limitations:** Documenting any simplifying assumptions or known limitations.

3. Survey of Algorithms of Comparable Complexity

3.1 Classical Algorithmic Paradigms

Algorithms are commonly classified by their **design paradigm** and complexity^[13]:

- **Greedy algorithms:** Make locally optimal choices at each step (e.g., Dijkstra's, Kruskal's).
- **Divide and conquer:** Break problems into subproblems, solve recursively, and combine (e.g., Merge Sort, Quick Sort).
- **Dynamic programming:** Solve overlapping subproblems with memoization (e.g., Knapsack, Floyd-Warshall).
- **Backtracking and branch-and-bound:** Systematic search for combinatorial problems (e.g., N-Queens, TSP).
- **Linear programming and optimization:** Mathematical optimization under linear constraints (e.g., Simplex, interior-point methods).
- **Randomized and approximation algorithms:** Use randomness or relax optimality for efficiency (e.g., Randomized Quick Sort, Approximate TSP).

Table 2: Comparison of Algorithmic Paradigms

Paradigm	Typical Complexity	Optimality	Example Problems
Greedy	$O(n \log n)$	May be suboptimal	MST, Huffman coding

Divide & Conquer	$O(n \log n)$	Optimal	Sorting, FFT
Dynamic Programming	$O(n^2)$ - $O(n^3)$	Optimal	LCS, Knapsack
Backtracking	Exponential	Optimal	SAT, N-Queens
Linear Programming	Polynomial	Optimal	Resource allocation
Randomized	Varies	Probabilistic	Quick Sort, Hashing

3.2 Modern and Hybrid Approaches

Recent years have seen the rise of **metaheuristics** (e.g., genetic algorithms, swarm intelligence), **machine learning-based algorithms**, and **quantum algorithms**. For example, swarm intelligence algorithms such as Particle Swarm Optimization (PSO), Cat Swarm Optimization (CSO), and Artificial Lizard Search Optimization (ALSO) have demonstrated superior performance on complex, high-dimensional optimization problems^[14].

Quantum algorithms (e.g., Shor's, Grover's, QAOA, VQE) offer exponential or quadratic speedups for specific problems, but are currently limited by hardware constraints and problem applicability^[15].

Table 3: Recent Benchmark Results for Swarm Intelligence Algorithms (2025)

Algorithm	Accuracy (RMSE < 0.05)	Avg. Execution Time	Complexity
ALSO	93%	Low	$O(nm)$
CSO	93%	Low	$O(nm)$
PSO	90%	Moderate	$O(nm)$
FA	51%	High	$O(nm^2)$

Source: Warnakulasooriya & Segev, 2025^[14]

4. Design Methodologies and Architecture Patterns

4.1 Top-Down and Bottom-Up Design

Top-down design decomposes a complex problem into smaller subproblems, refining each until they are manageable. This approach is well-suited for modular, maintainable systems and is often visualized with structure diagrams or flowcharts^[16].

Bottom-up design builds solutions from well-understood components, integrating them into larger systems. This is common in data-driven and machine learning pipelines.

4.2 Modularization and Abstraction

Modern algorithmic systems emphasize **modularity**, **abstraction**, and **separation of concerns**. Abstract Data Types (ADTs) and interfaces allow for interchangeable components and facilitate testing and verification^[16].

- **Python:** Rapid prototyping, rich libraries (NumPy, scikit-learn, TensorFlow), dominant in data science and ML.
- **Java:** Portability, strong type safety, enterprise applications.
- **Go, Rust:** Modern concurrency, safety, and performance features.
- **JavaScript:** Web-based algorithm visualization.
- **Julia, R:** Scientific computing and statistics.

Table 4: Language Strengths for Algorithm Implementation

Language	Strengths	Weaknesses	Best Use Cases
C++	Speed, STL, low-level	Steep learning curve	Competitive programming
Python	Readability, libraries	Slower execution	ML, rapid prototyping
Java	Portability, reliability	Verbose syntax	Production systems
Go	Concurrency, simplicity	Smaller ecosystem	Distributed systems
Rust	Safety, performance	Learning curve	Systems, parallel algorithms

6.2 Libraries and Frameworks

- **Numerical and scientific computing:** NumPy, SciPy, Eigen, BLAS/LAPACK
- **Machine learning:** scikit-learn, TensorFlow, PyTorch, XGBoost
- **Optimization:** CVXPY, Gurobi, CPLEX, OR-Tools
- **Visualization:** Matplotlib, Seaborn, D3.js
- **Testing:** pytest, unittest, QuickCheck (property-based testing)^[24]

6.3 Tooling for Reproducibility

- **Version control:** Git, GitHub, GitLab
- **Environment management:** Conda, Docker, virtualenv
- **Notebook environments:** Jupyter, Colab^[26]
- **Continuous integration:** GitHub Actions, Jenkins

7. Data Requirements and Preprocessing Pipelines

7.1 Data Collection and Management

High-quality data is essential for algorithmic performance, especially in ML and data-driven algorithms. Key considerations include:

- **Data provenance and documentation**
- **Handling missing values and outliers**
- **Ensuring data privacy and compliance**

7.2 Preprocessing and Feature Engineering

A robust preprocessing pipeline includes:

1. **Data cleaning:** Error detection, type correction
2. **Missing-value imputation:** Mean/median, k-NN, model-based
3. **Categorical encoding:** Label encoding, one-hot, feature hashing^[18]
4. **Scaling/normalization:** Standardization, min-max scaling
5. **Feature construction and selection:** Manual, automated (AutoML), filter/wrapper/embedded methods
6. **Dimensionality reduction:** PCA, autoencoders, t-SNE, UMAP

Figure 1: Canonical Preprocessing Pipeline

Data Splitting □ Imputation □ Encoding □ Scaling □ Feature Construction □ Feature Selection □ Model Training

Automation tools (e.g., scikit-learn Pipelines, PyCaret, Featuretools) are widely used for reproducibility and efficiency^[18].

8. Testing, Verification, and Validation Techniques

8.1 Unit and Integration Testing

- **Unit tests:** Verify individual components.
- **Integration tests:** Ensure correct interaction between modules.

8.2 Property-Based and Invariant Testing

Property-based testing (e.g., QuickCheck) automatically generates test cases to verify that properties (invariants) hold for a wide range of inputs^[24].

8.3 Formal Verification

Formal methods (e.g., Hoare logic, model checking) are increasingly used for critical algorithms, especially in safety- and security-sensitive domains^[20].

8.4 Benchmarking and Performance Evaluation

Algorithm benchmarking involves systematic evaluation against standard datasets and metrics^[29].

- **Performance metrics:** Execution time, memory usage, accuracy, scalability

- **Test data:** Representative, real-world datasets
- **Baseline comparison:** Against industry-standard or state-of-the-art algorithms
- **Environment setup:** Standardized hardware/software configurations
- **Repeatability:** Ensuring results are reproducible

Table 5: Example Benchmarking Metrics

Metric	Description	Typical Tools
Execution time	Time to complete task	time, cProfile, JMH
Memory usage	Peak and average memory consumption	valgrind, memory_profiler
Accuracy	Correctness of output	Custom, scikit-learn
Scalability	Performance as data size increases	Custom scripts

9. Performance Optimization and Complexity Analysis

9.1 Complexity Analysis

- **Time complexity:** Big O notation (e.g., $O(n)$, $O(n \log n)$, $O(n^2)$)
- **Space complexity:** Memory usage as a function of input size

Amortized analysis is used for operations with variable costs (e.g., dynamic arrays, hash tables) [31].

9.2 Profiling and Bottleneck Identification

Profiling tools (e.g., cProfile, gprof, perf) help identify performance bottlenecks.

9.3 Parallelization and Distributed Computing

- **Parallel algorithms:** Multi-threading, SIMD, GPU acceleration
- **Distributed algorithms:** MapReduce, Spark, MPI
- **Benchmarking parallel vs. serial implementations:** Multi-language suites compare performance across C, C++, Go, Java, Python, Rust^[22].

9.4 Hardware Acceleration

- **FPGAs, ASICs:** For specialized, high-throughput applications (see SODA-Benchmarks)^[29].
- **Cloud and edge deployment:** For scalable, low-latency applications.

10. Security, Privacy, and Adversarial Considerations

10.1 Security

- **Input validation and sanitization**
- **Defensive programming**
- **Formal verification for critical code**

10.2 Privacy

- **Differential privacy:** Ensuring outputs do not leak sensitive information.
- **Federated learning:** Training models without centralizing data.

10.3 Adversarial Robustness

- **Adversarial attacks:** Inputs crafted to fool models (e.g., in deep learning)^[8].
 - **Defenses:** Adversarial training, verification, explainability.
-

11. Reproducibility, Licensing, and Intellectual Property

11.1 Reproducibility

- **Versioned code and data**
- **Environment capture (Docker, Conda)**
- **Experiment tracking (MLflow, DVC)**
- **Open benchmarks and datasets**

11.2 Licensing and IP

- **Clear documentation of license and IP status**
 - **Compliance with open-source and proprietary components**
 - **Patent considerations for novel algorithms**
-

12. Ethical Considerations and Societal Impact

12.1 Fairness, Accountability, and Transparency

- **Bias detection and mitigation:** Ensuring algorithms do not perpetuate or amplify societal biases.

- **Explainable AI (XAI):** Techniques such as SHAP, LIME, and ELI5 provide interpretable explanations for model decisions^[10].
- **Accountability frameworks:** Documenting decision-making processes and enabling audits.

12.2 Societal Impact

- **Job displacement and workforce transformation**
 - **Environmental impact (e.g., energy usage of large models)**
 - **Regulatory compliance and public trust**
-

13. Documentation, Reporting, and Deliverables

13.1 Documentation Standards

- **Comprehensive API documentation**
- **Inline code comments and docstrings**
- **Design and architecture diagrams**
- **User and developer guides**

13.2 Reporting

- **Benchmark reports:** Detailed performance and accuracy metrics
 - **Test coverage and validation results**
 - **Compliance and risk assessment documentation**
-

14. Case Studies and Examples

14.1 End-to-End Algorithm Development: Parking Assistance System

A recent SAE technical paper describes an **end-to-end parking algorithm development pipeline** that integrates procedural scenario generation, expert control, offline optimization, and real-world deployment^[17]. Key steps include:

1. **Synthetic data generation:** Automated creation of diverse parking scenarios using Blender.
2. **Trajectory generation:** Expert controllers and optimization algorithms produce reference trajectories.
3. **Data rendering:** High-fidelity images and labels for training.
4. **Algorithm training:** End-to-end learning of parking maneuvers.
5. **Deployment and validation:** Real-world testing in vehicles.

This approach exemplifies the integration of simulation, automation, and real-world feedback in modern algorithm development.

14.2 Machine Learning Pipeline: California Housing Regression

A Jupyter notebook demonstrates a **regression pipeline** for predicting California housing prices [26]:

- **Data loading and exploration**
- **Preprocessing:** Feature scaling, train/test split
- **Model training:** Linear regression
- **Evaluation:** Mean squared error, R2 score
- **Model serialization:** Saving trained model for deployment

This example illustrates best practices in data-driven algorithm development, including reproducibility and interpretability.

15. Quantum vs. Classical Algorithms: A Comparative Perspective

Quantum algorithms offer **theoretical speedups** for specific problems (e.g., Shor's for factoring, Grover's for search), but are currently limited by hardware and applicability^[15]. Classical algorithms remain dominant for most practical applications, but hybrid quantum-classical approaches are an active research area.

Table 6: Quantum vs. Classical Algorithm Comparison

Problem	Classical Complexity	Quantum Complexity	Status
Integer Factor.	Exponential	Polynomial (Shor)	Quantum wins
Unstructured Search	$O(N)$	$O(\sqrt{N})$ (Grover)	Quantum wins
Optimization	Varies	Hybrid (QAOA, VQE)	Research phase
ML/AI	Varies	Quantum ML	Early research

16. Deployment, Scaling, and Operationalization

16.1 Deployment Pipelines

- **Containerization:** Docker, Kubernetes for scalable deployment^[33]
- **CI/CD:** Automated testing and deployment
- **Monitoring and observability:** Prometheus, Grafana, Jaeger

16.2 Scaling Strategies

- **Horizontal scaling:** Adding more instances (pods, nodes)
- **Vertical scaling:** Increasing resources per instance
- **Autoscaling:** Dynamic adjustment based on load (HPA, KEDA)

- **Service mesh:** Advanced traffic management (Istio)

Figure 2: Kubernetes Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-container
          image: myapp-image:latest
      ports:
        - containerPort: 80
```

17. Human Factors: Interpretability and User Interfaces

17.1 Explainability

Explainable AI (XAI) is now a requirement in regulated domains. Techniques include:

- **Feature importance:** SHAP, LIME, ELI5
- **Visualization:** Heatmaps, decision trees, counterfactuals
- **Natural language explanations:** For end-user interpretability

17.2 User Interfaces

- **Interactive dashboards:** For monitoring and control
 - **APIs and SDKs:** For integration with other systems
 - **Documentation and tutorials:** For onboarding and support
-

18. Recent Academic Literature and Trends (2024-2026)

- **Automated algorithm generation:** LLM-driven frameworks for end-to-end code synthesis and optimization^[34]
 - **Swarm intelligence and metaheuristics:** Continued advances in accuracy and efficiency for complex optimization^[14]
 - **Quantum-classical hybrid algorithms:** Early-stage research and benchmarking^[15]
 - **Explainable AI:** New methods for transparency, fairness, and regulatory compliance^[10]
 - **Benchmarking and reproducibility:** Standardized suites and open datasets (e.g., SODA-Benchmarks)^[28]
 - **Ethics and governance:** Integration of ethical frameworks and risk management into algorithmic pipelines^[6]
-

19. Benchmarks, Datasets, and Evaluation Suites

- **Standard datasets:** UCI, OpenML, Kaggle, domain-specific repositories
 - **Benchmark suites:** SODA-Benchmarks for hardware/software co-design, MLPerf for machine learning, custom suites for optimization and search^[28]
 - **Evaluation metrics:** Accuracy, precision, recall, F1, ROC-AUC, RMSE, latency, throughput
-

20. Implementation Notes and Reproducible Code Templates

20.1 LaTeX for Algorithm Specification

Professional documentation of algorithms often uses LaTeX with packages such as `algorithmic`, `algpseudocode`, and `algorithm2e` for clear, structured pseudocode and formal specification^[36].

Example: QuickSort in LaTeX (`algorithmicx`)

```
\begin{algorithm}
\caption{Quick Sort}
\begin{algorithmic}[1]
\Function{QuickSort}{$A$, low, high$}
\If{$low < high$}
\State $\text{pivotIndex} \leftarrow \text{Call}\{\text{Partition}\}(A, low, high)$
\State $\text{Call}\{\text{QuickSort}\}(A, low, \text{pivotIndex} - 1)$
\State $\text{Call}\{\text{QuickSort}\}(A, \text{pivotIndex} + 1, high)$
\EndIf
\EndFunction
\end{algorithmic}
\end{algorithm}
```

20.2 Python Code Template for Benchmarking

```
import time
import numpy as np

def benchmark_algorithm(algorithm, data, n_runs=10):
    times = []
    for _ in range(n_runs):
        start = time.time()
        algorithm(data.copy())
        times.append(time.time() - start)
    print(f"Average time: {np.mean(times):.4f}s ± {np.std(times):.4f}s")
```

Conclusion

The **end-to-end development of professional algorithms** in 2026 is a multidisciplinary endeavor, integrating advances in mathematical modeling, software engineering, data science, ethics, and deployment. The process is characterized by rigorous requirements engineering, formal specification, modular and automated design, comprehensive testing and benchmarking, and a strong emphasis on reproducibility, explainability, and societal impact. As quantum computing, AI, and automation continue to reshape the field, the principles and best practices outlined in this report provide a robust foundation for innovation, compliance, and responsible deployment.

Key Takeaways:

- **Legal and ethical compliance** are integral to algorithmic innovation.
 - **Benchmarking and reproducibility** are essential for trust and progress.
 - **Explainability and fairness** are now mandatory in many domains.
 - **Automation and end-to-end pipelines** drive efficiency and scalability.
 - **Quantum-classical synergy** is an emerging frontier.
 - **Open-source and collaborative development** accelerate advancement.
- By adhering to these principles and leveraging the latest tools and methodologies, professionals can design, implement, and deploy algorithms that are not only efficient and effective but also transparent, fair, and aligned with societal values.
-

References (36)

1. *Three routes to protecting AI systems and their algorithms under IP law*
<https://academic.oup.com/jiplp/article/16/3/247/6143561>

2. *Divide and Conquer vs. Dynamic Programming - Baeldung.*
<https://www.baeldung.com/cs/divide-and-conquer-vs-dynamic-programming>
3. *Comparative analysis of accuracy and computational complexity across 21*
<https://link.springer.com/article/10.1007/s12065-024-00997-6>
4. *Survey On Quantum Algorithms Vs Classical Algorithms: A Comparative*
<https://ijcrt.org/papers/IJCRT2411744.pdf>
5. *ALGORITHM SPECIFICATION.*
https://www.pvpsiddhartha.ac.in/dep_it/lecture%20notes/CDS/unit1.pdf
6. *Explainable Artificial Intelligence (XAI) - GeeksforGeeks.* <https://www.geeksforgeeks.org/artificial-intelligence/explainable-artificial-intelligencexai/>
7. *A Specification's Realm: Characterizing the Knowledge Required for*
<https://arxiv.org/html/2510.19853v1>
8. *A Practical Guide To Adversarial Robustness - Towards Data Science.*
<https://towardsdatascience.com/a-practical-guide-to-adversarial-robustness-ef2087062bec/>
9. *Ethics in AI: Why It Matters - professional.dce.harvard.edu.*
<https://professional.dce.harvard.edu/blog/ethics-in-ai-why-it-matters/>
10. *Best Open Source Licenses: A Comprehensive Guide for Developers and*
<https://dev.to/ashucommits/best-open-source-licenses-a-comprehensive-guide-for-developers-and-innovators-56mf>
11. *Is the law playing catch-up with AI? - Harvard Law School.* <https://hls.harvard.edu/today/is-the-law-playing-catch-up-with-ai/>
12. *Data Preprocessing and Feature Engineering for Data Mining ... - MDPI.*
<https://www.mdpi.com/2673-2688/6/10/257>
13. *3-hoare - CMU School of Computer Science.* <https://www.cs.cmu.edu/~aldrich/courses/654-sp09/notes/3-hoare-notes.pdf>
14. *sreekar2858/Multi_Language_Algorithms_Benchmark - GitHub.*
https://github.com/sreekar2858/Multi_Language_Algorithms_Benchmark
15. *An introduction to property-based testing with JUnit-Quickcheck.*
<https://www.ontestautomation.com/an-introduction-to-property-based-testing-with-junit-quickcheck/>
16. *machine-learning-examples.ipynb - Colab.* <https://colab.research.google.com/github/ubc-library-rc/intro-machine-learning/blob/main/Examples/machine-learning-examples.ipynb>
17. *2025-01-8060 : End-to-End Style Parking Algorithm Development Pipeline*
<https://www.sae.org/papers/end-end-style-parking-algorithm-development-pipeline-based-procedural-parking-scenario-synthetic-data-generation-2025-01-8060>
18. *Advanced DevOps Techniques: Scaling Microservices with Kubernetes.*
<https://www.nilebits.com/blog/2024/08/scaling-microservices-kubernetes/>
19. *arXiv:2503.10721v1 [cs.SE] 13 Mar 2025.* <https://arxiv.org/pdf/2503.10721>
20. *pnnl/soda-benchmarks (Software) .* <https://www.osti.gov/biblio/code-157726>
21. *6.046J Lecture 11: Amortized analysis - MIT OpenCourseWare.* <https://ocw.mit.edu/courses/6->

046j-design-and-analysis-of-algorithms-spring-

2012/83b82d45beb3776da72b7f3e1b3f42df_MIT6_046JS12_lec11.pdf

22. *Algorithm Benchmarking Methods* - *meegle.com*.

https://www.meegle.com/en_us/topics/algorithm/algorithm-benchmarking-methods

23. *Write Pseudo Code in LaTeX* . <https://www.baeldung.com/cs/latex-pseudocode-format-snippets>